

Recurrent Deep Sets: Predicting Weekly Gasoline Demand from Traffic Congestion

Luke O’Leary

August 5th, 2026

Abstract

I adapt and extend the Deep Sets architecture introduced by Zaheer et al. (2017) to learn embeddings for set-structured elements in a high-frequency time-series on their way to predicting the next time-step of a low-frequency series. This is particularly relevant in economics, where macro-level data is often reported on a weekly or monthly basis, while micro-level signals take place over irregular and more frequent intervals. Leveraging a dataset of 33 million traffic congestion events across 49 states over a six-year period, I attempt to predict the following week’s data print of motor gasoline product supplied from the current week’s traffic and weather activity. I show that recurrent Deep Sets outperform conventional recurrent autoencoder methods, with the final model specification achieving test predictions within 0.44 standard deviations of observed values.

1. A Real-Time Pulse of the Economy

“Almost every private company CEO that’s running his or her business are doing so with real-time information that isn’t subject to much revision, that is telling them what just happened at that very moment. There are normal long and variable lags in the conduct of monetary policy. What we’re really interested in is what’s happening right now. What we’re less interested in is echoes of history.” – Kevin Warsh, Chair of the U.S. Federal Reserve

Policymakers and financial markets have long depended on periodic data releases to gauge the health of the economy. Monthly reports of inflation, nonfarm payrolls, and GDP are among highly anticipated macroeconomic indicators that inform the Federal Reserve’s monetary policy. Similarly, public companies’ quarterly earnings calls reflect past performance to investors and offer a refined lens into different sectors. These releases move markets, particularly when new data deviates from expectations. However, their low-frequency means that important policy or investment decisions are rooted in lagged information that paints a sparse picture of the economic environment over time, when in fact economic activity is continually ongoing. Increasingly available unstructured data sources offer fingerprints of this activity at higher frequencies. Deep learning models are not only well-suited to handle unstructured data, but their high complexity creates a powerful tool for understanding more sophisticated underlying mechanisms in the economy.

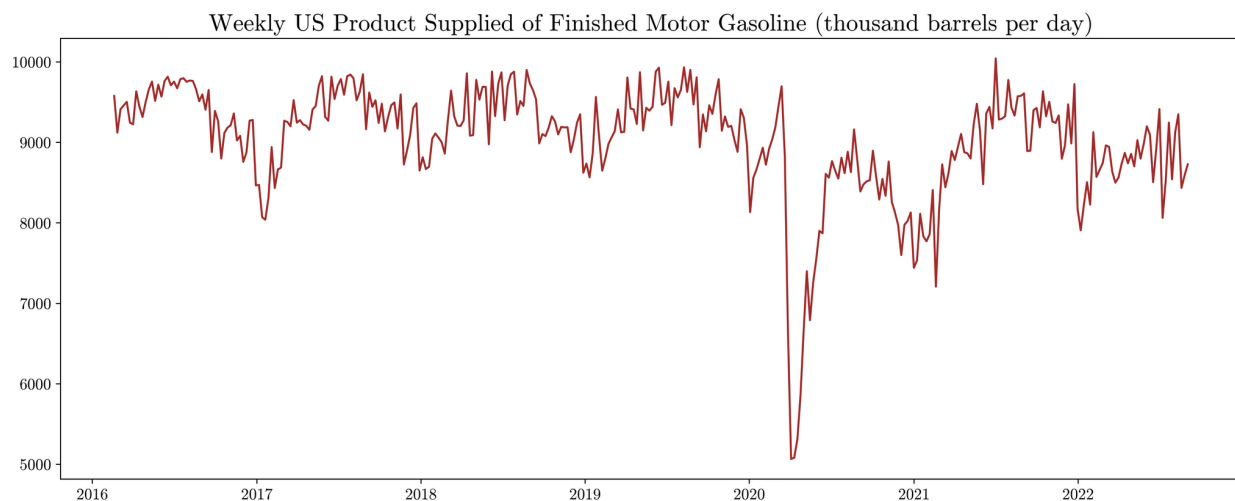
This project focuses on one of the primary data releases monitored by oil traders – the weekly petroleum status report published by the U.S. Energy Information Administration. Applying time-series methods, I target the larger question of **whether high-frequency data can accurately predict low-frequency economic prints.**

2. Dataset and Preprocessing Steps

Motor Gasoline Product Supplied

The U.S. Energy Information Administration (EIA) publishes weekly petroleum supply data each Wednesday at 10:30 AM EST. The report provides a summary of the prior week’s activity up to Friday at 7:00 AM EST, including stocks, net imports, and production of crude oil and petroleum products. I set out to predict the following week’s print of *finished motor gasoline product supplied*, which represents

the disappearance of gasoline from primary sources (e.g. refineries) and is how the EIA measures consumption. Traders who recognize signals of weakening gasoline demand may adjust their positions before price drops; while this only focuses on one side of the price equation, regional markets for gasoline face differing supply-side factors, such as import reliance, shipping routes, and refinery bottlenecks. On the other hand, gasoline consumption ultimately depends on how much people are driving. Furthermore, the gasoline market often drives the market for crude oil, given that oil is primarily used for passenger vehicle transportation.



Traffic Congestions

My input dataset contains approximately 33 million records of traffic congestion events across 49 states from February 2016 to September 2022. Each observation describes a traffic build-up, including its start and end time, miles of road impacted, delay compared to free traffic flow and typical traffic flow, its severity (reported by provider), congestion speed, and longitude-latitude coordinates. Weather data from the nearest airport weather station at the time of each event is also included, such as temperature, wind-chill, humidity, air pressure, visibility, windspeed, precipitation amount, and if a weather event (e.g. thunderstorm) was taking place. These records are a vantage into driving activity and gasoline demand across time. Traffic jams are not just a sign of more drivers on the road, but a car continues to burn fuel at rest, meaning consumers are drawing down greater amounts of gasoline over the course of their drive. Weather conditions also inform fuel consumption; people typically drive less during winter storms and greater during summer months, and fuel economy decreases in cold temperatures.

Preprocessing Steps

Traffic events are time-labelled, and when stacked chronologically, they construct a congestion history with each event represented as a vector. However, nationwide gasoline demand is not predicated on a single traffic jam happening in a specific location, but rather the prevalence of traffic throughout the country. Rather than analyzing events in silos, I construct a time series where each element is a coding for the nationwide traffic activity and weather variability in a given hour window. A week (beginning Friday at 7:00 AM EST) consists of 168 of these traffic-hour embeddings, which I use to predict the next EIA data print.

I split observations into training, validation, and test according to when they took place. The training set comprises the 255 weeks from February 12th, 2016, 7:00 AM EST to January 1st, 2021, 7:00 AM EST; all traffic events that began at any point within this timeframe are training observations. Each week has a corresponding EIA data print. The validation set contains traffic events and weekly gasoline product supplied figures for the 39 weeks from January 1st, 2021, 7:00 AM EST to October 1st, 2021, 7:00 AM EST and the test set covers the 48 weeks from October 1st, 2021, 7:00 AM EST to September 2nd, 2022, 7:00 AM EST.

From here, I removed redundant or irrelevant features for predicting gasoline demand. For instance, the traffic jam's longitude and latitude coordinates effectively compresses location information into two dimensions, so I excluded columns such as state and zip code. I also took out categorical variables that have many categories which add dimensionality to the data without adding prediction value (e.g. wind direction). From each event's start time and end time, I computed the duration of the traffic jam. Finally, I transformed the weather event column into six binary features (Rain, Snow, Fog, Thunderstorm, Hail, and Tornado) and converted congestion speed from categories to numeric values (preserving its ordinal structure).

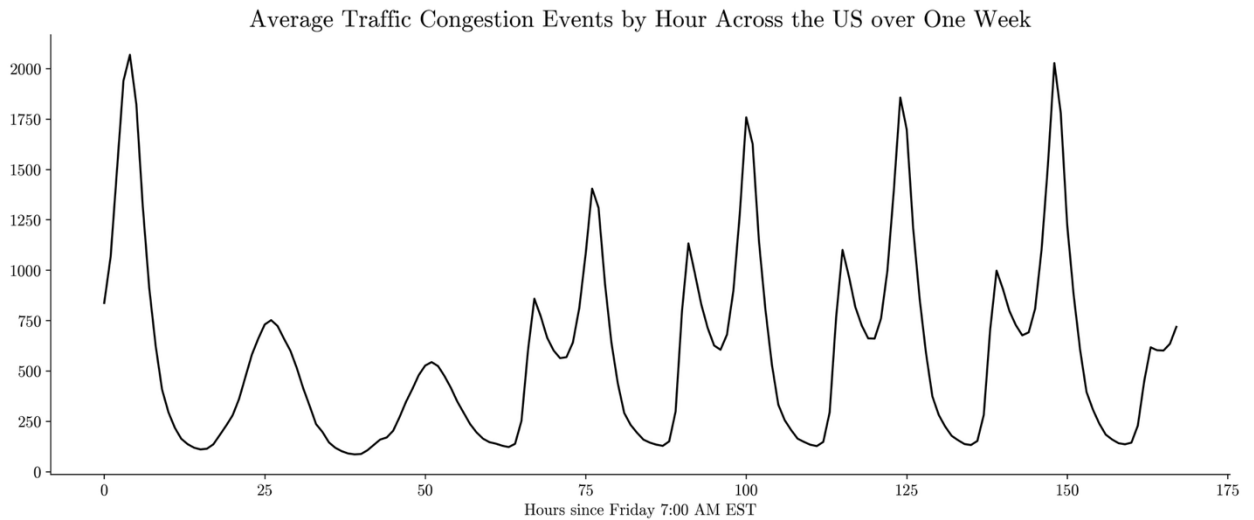
To construct traffic-hour embeddings, congestion events need to be mapped to the week and hour in which they occurred. The function *build_sets* (*start_date*, *end_date*) returns a data frame containing all one-hour periods and corresponding weeks within two dates, which I merge with my input dataset based on observations' start times.

Not all hours had a traffic event, so I appended an empty observation (containing 0s for all traffic fields and the column mean for all weather fields) to ensure no time step went missing. Missing numeric weather values were imputed with their column mean, and these were also among several variables containing extreme

outlier observations. Rather than dropping uncharacteristic observations outright, I set bounds for these variables and clipped values which fell beyond them.

These preprocessing steps were performed identically and independently for training, validation, and test datasets to ensure no leakage between splits. The output data and the continuous columns of the input data were then standardized relative to the training set mean and standard deviation.

3. Recurrent Deep Sets Model Design



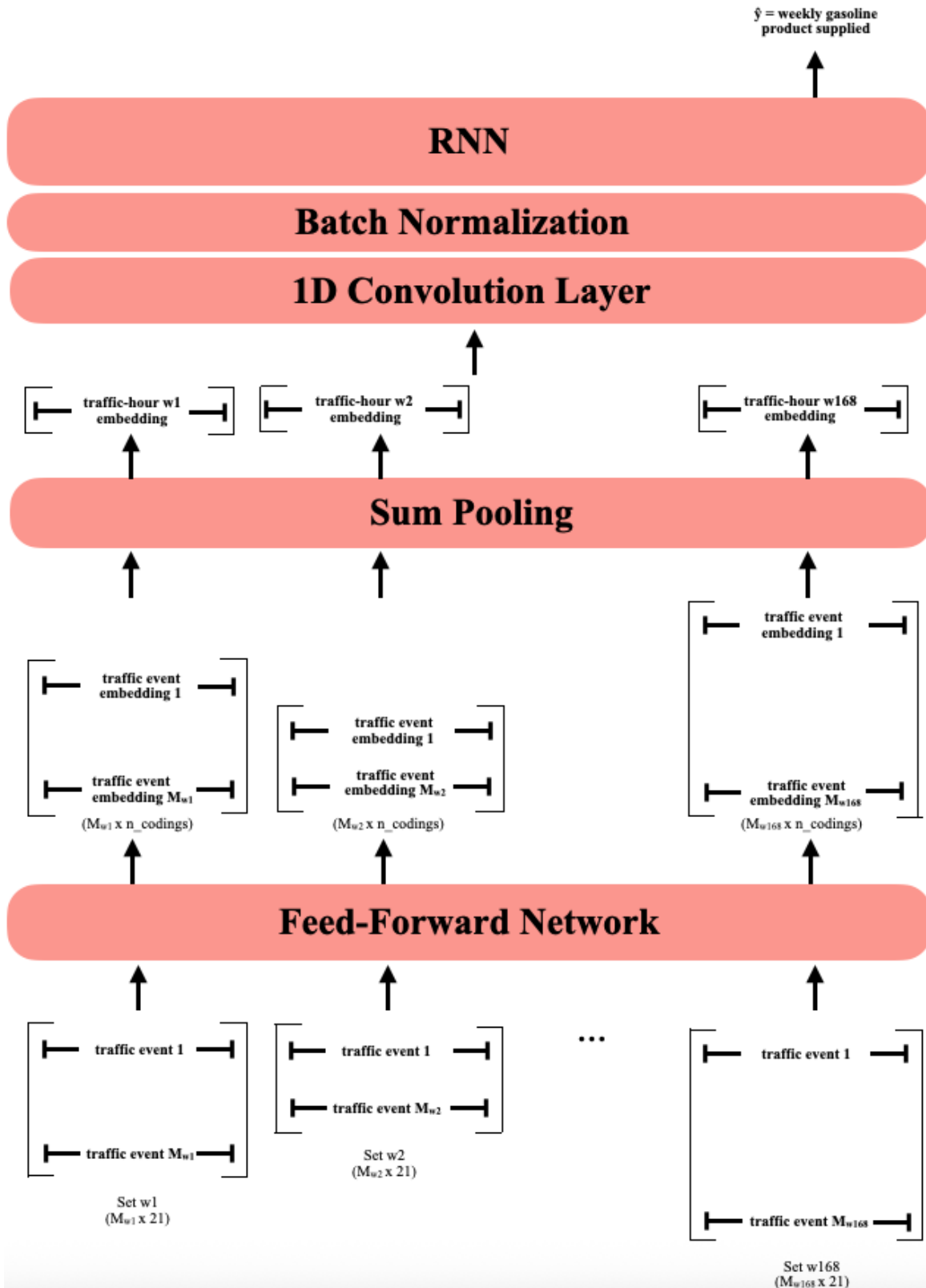
Set Input Problem

Traffic has clear time-dependencies, both over the course of a day and throughout the week. One can expect much fewer cars on the road in the U.S. at 4:00 AM EST (1:00 AM PST) compared to rush hour traffic at 5:00 PM EST (2:00 PM PST). Pileup also varies depending on if it's a weekday or the weekend. However, using conventional autoencoder techniques to learn the elements of this time series is challenged by the fact that the number of congestion incidents within each hour varies widely, meaning the model's input and output dimensions would not be fixed.

Each one-hour window comprised of traffic events makes up a *set* – a permutation-invariant tensor with varying dimensions. This implies that the ordering of events within the hour does not impact the hour's embedding. Drawing on Zaheer et al. (2017), I construct a deep sets model that learns traffic-hour codings before passing them through a recurrent neural network. This is an extension of the paper's initial

design which takes single-set inputs and passes codings into a feedforward neural network. My implementation learns codings for a series of sets in parallel.

Recurrent Deep Sets Architecture: This model adapts the Deep Sets design introduced by Zaheer et al. (2017) to learn embeddings for elements in time series.



Step 1: Traffic events for one week (i.e. comprising 168 sets) are fed *independently* into a feed-forward neural network that acts as an encoder.

For the network to process data in this manner, I wrote `to_tensor()`, which structures each week of traffic events as a 3D tensor of dimension $168 \times M_{wh} \times 21$, where M_{wh} denotes the number of congestion events which took place in hour h of week w . These ragged tensors are then padded along the second dimension up to `max_set_dim` – the maximum number of traffic events to take place within any hour of the training set – to ensure the tensor can be passed into the first feedforward layer.

Dense layers operate on the last tensor axis, so by inputting the 3D tensor, the network operates on each traffic event independently and in parallel while preserving which events belong to which hour in the week. The result is a 3D tensor with feature-space embeddings for each traffic event. The size of these codings (`n_codings`) is a hyperparameter which I initially set at 16 but later validate higher values.

Step 2: Aggregate traffic event embeddings using `sum()` pooling (i.e. *set pooling*).

Summing over the second axis (`tf.reduce_sum(axis=2)`) collapses the 3D tensors to 2D tensors, forming the 168 traffic-hour embeddings (each a vector of length `n_codings`) that define one week. Using `sum()` over alternative pooling operations (e.g. `max()`) ensures that we’re capturing both the frequency and severity of traffic pileup within the hour; for example, the traffic-hour embedding for an hour when many medium-sized pileups occurred throughout the country would have stronger signal than an hour when one large traffic jam occurred in one location. In our case where tensors are padded before passing them through the network, these extra rows would have little-to-no impact during aggregation.

Step 3: A 1D convolution layer contextualizes the traffic-hour embeddings amongst one-another.

The kernel size is set to 5, but padding and a filter count equal to `n_codings` preserves the sequence’s dimensionality. These inputs are batch-normalized to help combat exploding or vanishing gradients during training.

Step 4: Pass traffic-hour embeddings sequence through a recurrent neural network with many-to-one structure.

The recurrent network has one recurrent layer (with ReLU activation function) and one output layer that produces a scalar value (linear activation). The network is trained to minimize the mean squared error between this output and the gasoline

product supplied over the corresponding week. This design therefore bases predictions on traffic activity over the same period; for example, the prediction for product supplied from February 12, 7:00 AM, EST to February 19, 7:00 AM, EST is formed from traffic events which began in this window.

4. Model Training and Results

Hyperparameter Validation

Each model trains over 255 observations, where one forward pass involves one week of traffic activity. 3D tensors generated from *to_tensor()* are loaded into a TensorFlow dataset and are fed into the model with batch size equal to 4. I set the buffer size to 10 to ensure the order of the inputs changes over epochs, but purely random ordering (i.e. setting buffer size equal to sample size) is computationally intensive and slows training. All models were trained to minimize mean squared error loss between the actual gasoline product supplied and the predicted product supplied for the week corresponding to the input timeframe. Using mean absolute error to compare model performance reflects how far predictions deviate from actual amounts (after converting standardized values back to their original form).

I first validated hyperparameters tied to the architecture itself (i.e. recurrent layer time, neuron counts, and coding dimension) before exploring alternative optimizers.

Table 1: Validation Set Performance by Architecture

Recurrent Layer	Neurons	MAE (standardized)	MAE (thousand barrels per day)	Epochs
Simple RNN	3	0.7013	527.4	10
Simple RNN	5	0.6820	512.9	10
LSTM	32	0.6272	471.7	6
LSTM	64	0.6768	508.9	2
GRU	32	0.7148	537.5	9
GRU	64	0.6323	475.5	1

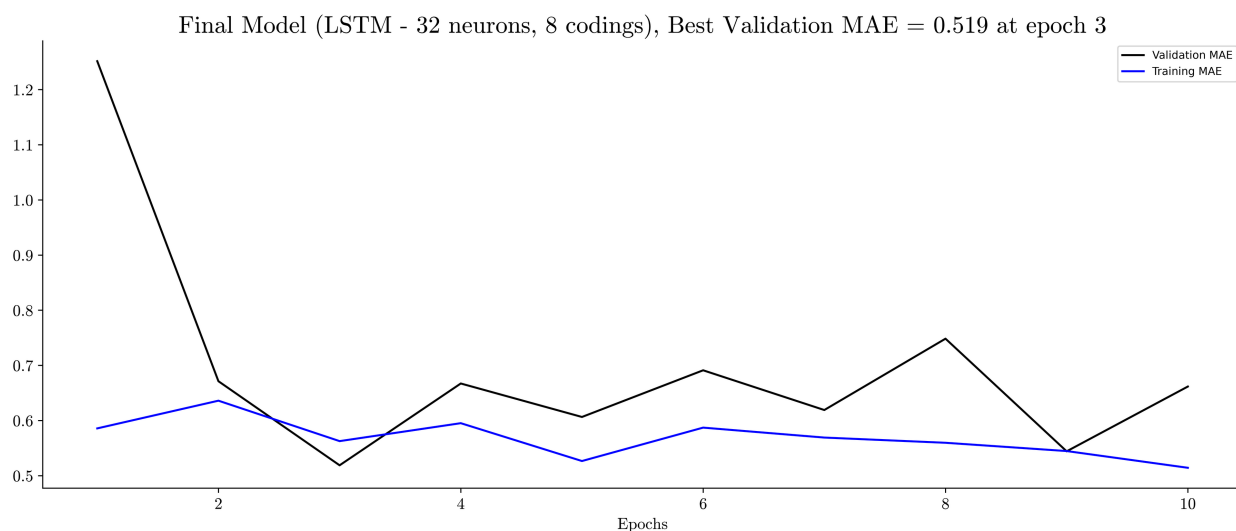
These six recurrent deep set models were trained over 10 epochs using Adam optimizer. Each architecture includes three feedforward layers (with ReLU activations) and sum() pooling that compresses input sets to 16-dimension traffic-hour embeddings ($n_{codings}=16$). The mean absolute error column indicates the lowest out-of-sample performance achieved and the corresponding epoch count.

The third model (LSTM with 32 neurons in each memory cell’s FC network) had the best out-of-sample performance. From here, I varied the size of the feature-space to see if there are gains from lower or higher-dimension traffic-hour codings, evaluating the model’s out-of-sample performance when $n_codings = 6$, $n_codings=8$ or when the coding layer is sparse ($n_codings=64$ with KL divergence regularization).

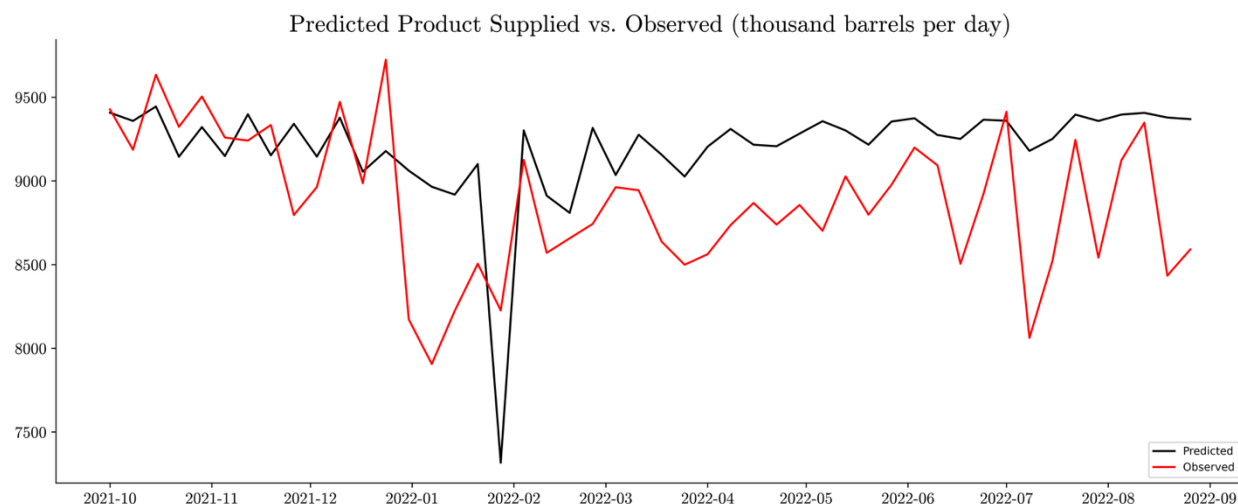
Table 2: Validation Set Performance – LSTM with 32 Neurons

Coding size	Sparse	Optimizer	MAE (standardized)	MAE (thousand barrels per day)	Epochs
4	No	Adam	0.5969	448.9	7
8	No	Adam	0.5188	390.1	3
16	No	Adam	0.6272	471.7	6
128*	Yes	Adam	0.6375	479.4	4
8	No	RMSProp	0.7368	554.0	10

**KL divergence targets 10 percent sparsity at the coding layer. The regularization penalty is 0.1.*

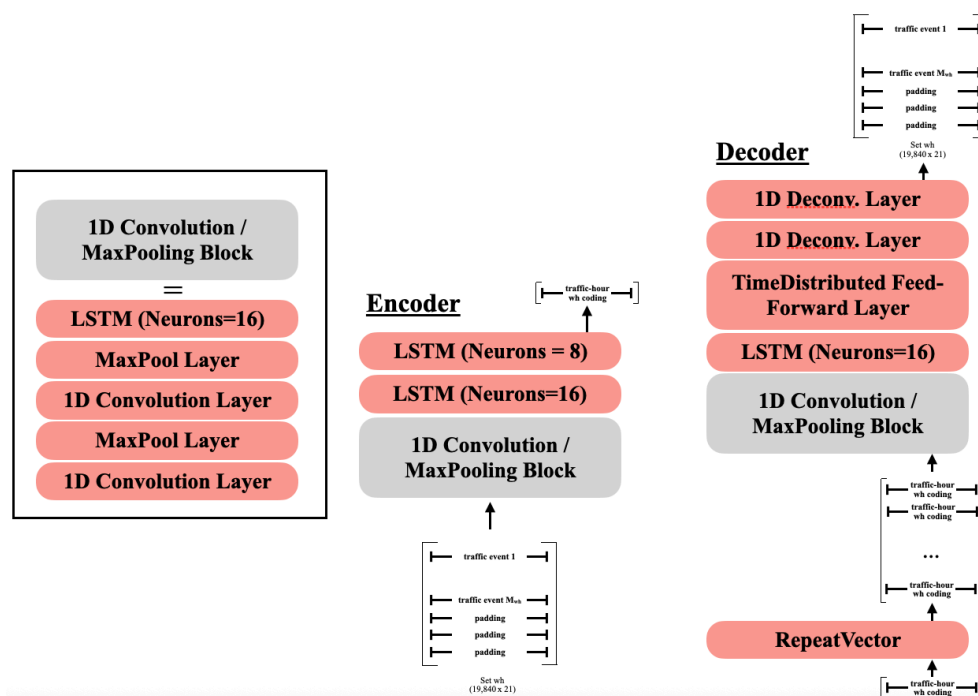


Test Performance



Recurrent Autoencoder

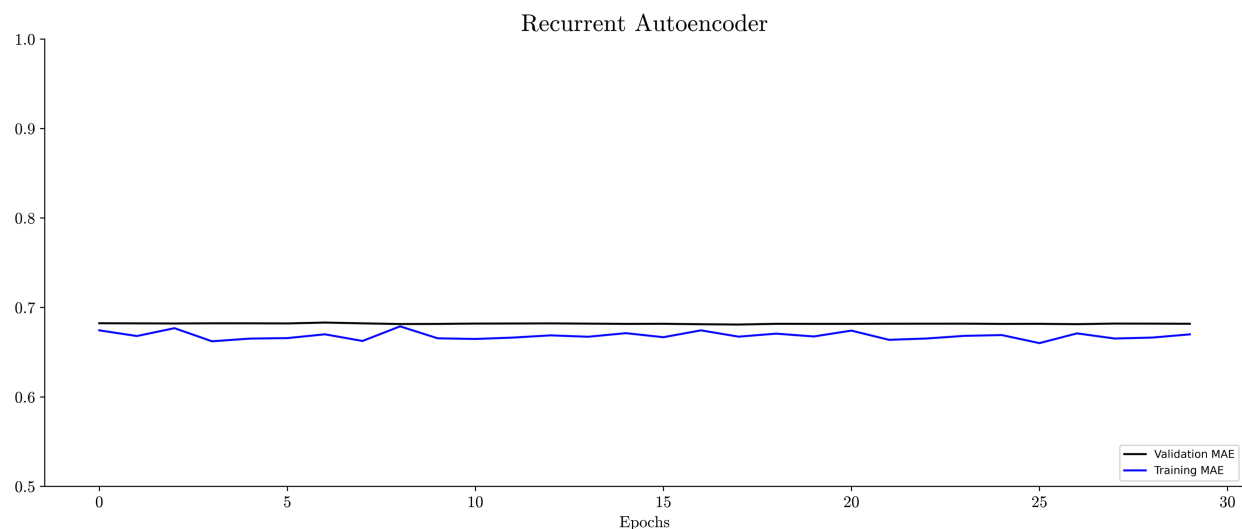
While autoencoders are constrained by fixed dimension input and output layers, one could pad the input data such that all observations are the same dimension. After training my final recurrent deep set model, I constructed a recurrent autoencoder and briefly explored this alternative approach.



My training set consists of 42,840 traffic-hours which I use to train my autoencoder. I wrote *to_padded_2D(max_set_dim)*, which converts the traffic-hours in my training set to padded 2D tensors. However, each input tensor has 19,840 rows – passing this directly into the network would grow the number of model parameters to an impractical level. Therefore, I add two 1D Convolution and 1D MaxPooling layers which reduce the input’s size along the first dimension. Given that observations are ordered chronologically within each set, this approach contextualizes traffic events based on when they occurred during the hour, whereas deep sets assumes permutation invariance.

Each MaxPooling layer has a stride of 8, thus reducing the input’s length to 310 before it is fed into the first LSTM layer. The model has one hidden LSTM layer with 16 neurons and one LSTM bottleneck layer that compresses the traffic-hour into an embedding vector of length 8. The first decoder layer repeats this coding as a 19,840-length sequence, which again requires two 1D Convolution and 1D MaxPooling rounds before being fed into the hidden LSTM layer. De-convolutions are performed at the end of the decoder to reconstruct the 2D tensor input.

After learning these alternative codings, I trained a recurrent neural network identical to the architecture used in my final deep sets model.



Tracking the training and validation loss over 30 epochs, the model struggles to learn the data-generating process that maps traffic activity with gasoline demand, which is likely due to the quality of the traffic-hour codings. The combination of sparsity induced by padding and aggressive dimensionality reduction via MaxPooling prevents the autoencoder from learning the latent features of each

traffic-hour. In addition, processing these codings independently ignores time-dependencies across hours in the day and week.

5. Conclusion

Branching away from typical autoencoder techniques, I developed a unique approach for coding traffic-hours which captures time-dependencies over the course of the day and week. One limitation of this design is the amount of memory required during training. Because Dense layers in Keras are unable to process ragged tensors, I had to pad these tensors according to the largest set in my data. Thus, while the number of parameters in the model is not exceptionally high, the number of computations performed in each forward-pass slows down training heavily, with each model specification taking roughly 2 hours to fit over 10 epochs.

While the final model predictions are not accurate enough to make policy or trading decisions on at this stage, this is a useful framework that can be carried over into other “many-to-one” time-series contexts. Traffic and weather are two prominent determinants of gasoline demand, but this model does not account for other factors such as improvements in fuel efficiency over time and gasoline prices. A more comprehensive dataset may enrich the hour codings and enhance their predictive power.

6. References

- Lee, J., Lee, Y., Kim, J., Kosiosek, A., Choi, S., & Teh, Y. W. (2019, May). Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning* (pp. 3744-3753). PMLR.
- Moosavi, S., Samavatian, M. H., Nandi, A., Parthasarathy, S., & Ramnath, R. (2019, July). Short and long-term pattern discovery over large-scale geospatiotemporal data. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 2905-2913).
- U.S. Energy Information Administration (n.d.). *Weekly Petroleum Status Report*. <https://www.eia.gov/petroleum/supply/weekly>
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., & Smola, A. J. (2017). Deep sets. *Advances in neural information processing systems*, 30.